Sparse Event-Driven Neuromorphic Decoders for Implantable BCIs

Manraj Mondair
Stanford University
manraj@stanford.edu

Alexander Soto
Stanford University
alexsoto@stanford.edu

Abstract

Implantable brain–computer interfaces (BCIs) are constrained far more by energy and bandwidth than by raw decoding accuracy: every spike transmitted and every multiply–accumulate performed must dissipate almost nothing on a device that sits inside cortex. This motivates *event-driven* spiking decoders that process only a fraction of the spike stream. We ask two questions that are usually entangled in the spiking-decoder literature—*how much spike information is kept* and *how a decoder exploits temporal structure*—and measure their separate effect on a standard continuous-velocity decode of primary motor cortex (Neural Latents Benchmark MC_RTT), alongside the energy each design would consume on neuromorphic hardware. Across a multi-seed, blocked cross-validation grid we find that temporal context, not fine within-bin spike timing, carries the signal. A memoryless decoder reaches only $R^2 \approx 0.17$; $\approx 200\,\mathrm{ms}$ of history triples it to 0.51–0.54, and at that matched window a linear decoder, a fixed random-projection reservoir spiking neural network (SNN), and a backpropagation-trained leaky integrate-and-fire (LIF) SNN are statistically indistinguishable. A battery of null controls shows the decode is driven by per-bin firing rate and its temporal alignment to movement—not within-bin spike order, precise timing, or neuron identity. Given each decoder its own validation-selected context window, the spiking decoders pull ahead: a trained LIF SNN whose linear readout sees a lag-stacked window of its per-bin hidden activity reaches $R^2 = 0.68$ at full budget—the most accurate decoder at every event budget—with a fixed-encoder reservoir SNN (0.64) just behind, both in the latent-dynamics tier of the public leaderboard. The SNNs therefore lead on *both* accuracy and energy: their sparse, event-driven synaptic operations cost tens of nanojoules per prediction on Loihi-2-class hardware—40–$500\times$ below a dense CPU readout of the same workload. We release the full pipeline, controls, and a $32\times$H100 reproducibility harness.

1 Introduction

A clinical-grade implantable BCI must continuously decode intended movement from intracortical spiking while dissipating only a few milliwatts: heat above $\sim 1\,^{\circ}\mathrm{C}$ damages cortical tissue, and a fully implanted device has neither a fan nor a wired power supply [3, 4, 5]. The two dominant costs are (i) *bandwidth*—the energy of digitizing and moving spike data off the array—and (ii) *computation*—the multiply–accumulate (MAC) operations the decoder performs per prediction. Both scale with how much of the spike stream the decoder touches. This is precisely the regime in which *neuromorphic*, event-driven computation is attractive: a spiking decoder that updates its state only when a spike arrives can, in principle, skip the bulk of the dense arithmetic a conventional readout performs, and map the rest onto sub-picojoule synaptic operations on chips such as Intel Loihi 2 or IBM NorthPole [16, 17, 18].

The case for spiking decoders is, however, usually argued by entangling two very different claims. The first is an *information* claim: that fine, sub-bin spike timing carries motor information that rate-based decoders discard, so a timing-sensitive spiking model should be more accurate. The second is an *efficiency* claim: that event-driven execution is cheaper. These are routinely reported together, on different datasets and decoders, which makes it hard to know whether a spiking model wins because of timing, because of temporal context, or merely because it was tuned harder than its baseline. We disentangle them on a single, standard benchmark.

We study continuous 2-D cursor-velocity decoding from primary motor cortex (M1) on the Neural Latents Benchmark MC_RTT dataset [1, 2], and we manipulate two factors independently. The first is an *event budget* f: for each $50\,\mathrm{ms}$ bin we retain only the earliest $\max(1, \lfloor f \cdot n \rfloor)$ of its n spike events and rebuild the rate code from the survivors, so every decoder sees a consistently sparsified stream as $f \to 0$. The second is the *temporal context* each decoder is allowed to use, from a strictly memoryless single-bin readout up to $\sim 1.4\,\mathrm{s}$ of history. Crossing these two axes, under blocked cross-validation with multi-seed error bars and a battery of structure-destroying null controls, isolates what actually drives the decode—and lets us cost the resulting models on four hardware targets with their *measured* synaptic-operation counts rather than a nominal FLOP estimate.

Contributions.

1. **Temporal context, not spike timing, is the dominant lever.** A memoryless decoder reaches $R^2 \approx 0.17$; stacking $\approx 200\,\mathrm{ms}$ of history triples it to 0.51–0.54 for the linear, reservoir-SNN, and trained-SNN decoders alike, which become statistically indistinguishable once context is matched (Sec. 5.1, 5.2).
2. **A controlled null battery pins down the carrier of the decode.** Destroying bin-to-velocity alignment (circular shift) collapses R^2 to chance, while shuffling within-bin spike order, randomizing precise spike times, or permuting neuron identity leaves it essentially unchanged (Sec. 5.3). The decode rides on per-bin rate and its temporal alignment to movement.
3. **Given deep context, neuromorphic decoders are the most accurate, not merely the most efficient.** A trained LIF SNN that routes history to its *readout*—rather than its saturating membrane—reaches $R^2 = 0.68$ at full budget and leads at every event budget, with a fixed random-projection reservoir SNN just behind (Sec. 5.4, 5.5).
4. **The neuromorphic payoff is energy, quantified from measured synaptic operations.** The event-driven readout avoids 79–97% of the dense MACs, and the trained SNN costs tens of nanojoules per prediction on Loihi-2-class hardware—40–$500\times$ below a CPU evaluation of the same model (Sec. 5.6).
5. **A reproducible, audited pipeline** with leakage-safe splits, twelve null/ablation analyses, and a cluster harness, released in full (Sec. 8).

2 Related work

Motor decoding and the Neural Latents Benchmark. Continuous velocity decoding from M1 has a long history, from population-vector and Kalman-filter decoders [6, 4] to modern latent-variable models. The Neural Latents Benchmark (NLB) standardizes this on a small set of recordings, including MC_RTT, a self-paced random-target reaching session in monkey M1 [1, 2]. The leaderboard is topped by latent dynamical-systems models—LFADS and AutoLFADS [7, 8], the Neural Data Transformer (NDT) [9], and MINT [10]—which reach velocity $R^2 \approx 0.62$–0.69, while strong linear, GPFA, and switching-LDS baselines sit at 0.49–0.58. We use MC_RTT as a common substrate on which every decoder is held to identical splits and metrics, and we position our results against this published tier (Sec. 5.9).

Spiking decoders for brain–machine interfaces. SNN decoders for BMI date back to analog VLSI Kalman implementations [19] and have re-emerged with surrogate-gradient training and dedicated silicon: sparsity-aware SNN decoders on RISC-V microcontrollers [20], head-to-head ANN-vs-SNN decoding studies [21], and hybrid neuromorphic decoders [22]. These works convincingly demonstrate *deployability*, but typically report a single operating point and conflate the information and efficiency arguments. Our contribution is methodological: we vary the event budget and

the temporal-context budget independently and run the same null controls on every decoder, so the accuracy comparison is causal rather than incidental.

Surrogate-gradient SNNs and reservoirs. We train LIF networks end-to-end with the fast-sigmoid surrogate gradient [11, 12, 13], and compare against a fixed random-projection LIF *reservoir* in the liquid-state / echo-state tradition [14, 15], where only a linear readout is fit. Contrasting a trained encoder against a fixed one, at matched context, directly tests whether end-to-end training of the spiking encoder is what matters—or whether the readout's temporal window is.

Neuromorphic energy. Per-synaptic-operation energy figures for Loihi 2 ($\approx 23\,\mathrm{pJ}$) and NorthPole INT8 ($\approx 2\,\mathrm{pJ}$) [16, 17, 18], set against a dense CPU MAC ($\approx 1\,\mathrm{nJ}$) and an A100 ($\approx 30\,\mathrm{pJ}$), let us convert measured operation counts into arithmetic energy. We report energy only as arithmetic energy (no memory, leakage, or radio), which is the conservative lower bound the neuromorphic argument should be held to.

3 Dataset and problem setup

MC_RTT (DANDI 000129) records $N = 98$ M1 units during continuous self-paced reaching, with simultaneous cursor kinematics. We bin spikes at $50\,\mathrm{ms}$ into a dense $[T \times N]$ count matrix *and* per-bin sparse (time, neuron) event lists with within-bin times in milliseconds, sorted ascending; the two representations are kept exactly consistent so the event budget and the rate code never disagree. After binning, $T = 12{,}982$ usable bins ($\approx 11\,\mathrm{min}$) remain. The decoding target is 2-D cursor velocity $v_t = (v_{x,t}, v_{y,t})$, computed by centered finite difference of cursor position and smoothed with a $\sigma = 1$-bin ($50\,\mathrm{ms}$) Gaussian along time, following standard BCI practice; an ablation confirms the conclusions are unchanged at $\sigma \in \{0, 1, 2\}$ bins (App. A.1).

Leakage-safe evaluation. Splits are time-contiguous ($70/15/15$ train/val/test) rather than random, because nearby bins are correlated and a shuffled split would leak. A one-bin *boundary gap* is dropped on each side of every split boundary so a 3-point finite-difference velocity at a training bin can never depend on positions from validation or test. Lag and history features are zero-padded across split boundaries by construction, so a test-bin feature never reaches back into the training block. All decoders train, validate, and score on these identical indices and on the identical joint R^2 (Sec. 4.5), so every comparison in the paper is apples-to-apples.

4 Methods

4.1 Event budget

For a fraction $f \in (0, 1]$, each bin keeps its earliest $k = \max(1, \lfloor f \cdot n \rfloor)$ spike events; the dense counts are rebuilt from the survivors so that every decoder—rate-based or spiking—sees the same sparsified stream. Because the event list is sorted, the filter is a head truncation that preserves the strict-monotonicity invariant the spiking encoders rely on. Retained event totals fall from 256,787 at $f = 1.0$ (19.8/bin) to 21,204 at $f = 0.10$ (1.6/bin). A separate control (App. A.1) replaces "earliest k" with "random k" and "latest k"; their near-equality establishes that the budget probes *how much rate survives*, not a privileged early-spike timing channel.

4.2 Decoders

All decoders predict $\hat{v}_t \in \mathbb{R}^2$. Regularization, history depth, and (for the SNNs) firing threshold are selected on the validation split only.

- **Ridge (counts).** An L_2-regularized linear map from the per-bin count vector; α chosen on validation. The memoryless reference point.
- **Ridge + history.** The same, with the previous k bins stacked as features (boundary-safe); k selected on validation. The strong linear baseline.

- **First-spike latency.** Per-neuron time-to-first-spike (silent neurons at a $50\,\mathrm{ms}$ sentinel) $\rightarrow$ ridge readout. A pure within-bin-*timing* control: if latency carried information beyond rate, this would beat the count ridge.
- **Reservoir SNN.** A fixed (untrained) random-projection LIF encoder is driven by the within-bin events in spike-time order, leaking between events at the true inter-spike interval $e^{-\Delta t/\tau}$ and hard-resetting on threshold crossing. Only a ridge readout over the per-bin hidden spike counts—optionally lag-stacked with a boundary-safe history window—is fit.
- **Trained LIF SNN.** An end-to-end LIF network. Each $50\,\mathrm{ms}$ bin is re-binned into 10 sub-bins of $5\,\mathrm{ms}$; the hidden membrane evolves as

$$u_t = u_{t-1}\, e^{-\Delta/\tau} + W x_t, \qquad s_t = \mathbb{1}[\, u_t \geq \theta\,], \qquad u_t \leftarrow u_t - s_t\,\theta, \tag{1}$$

  with input projection W, threshold θ, and per-bin hidden spike count $z = \sum_t s_t$ passed to a linear velocity readout. All weights (W, readout) are trained by Adam with backpropagation-through-time and the fast-sigmoid surrogate gradient $\partial s/\partial u = \sigma\,(1 + \sigma|u-\theta|)^{-2}$ [11]. Crucially, deep history reaches the readout by *lag-stacking the trained per-bin hidden features* (a `readout_lag` window), not by lengthening the input sequence (Sec. 5.5).
- **Deeper SNN.** A multi-layer (optionally recurrent) LIF with a lag-stacked readout—a capacity probe to test whether the remaining headroom is model capacity or temporal context.

4.3 Null controls

To identify what the decode actually rides on, we build null distributions by re-fitting a decoder on inputs that each destroy one kind of structure: (i) *order shuffle*—permute the $(\text{time}, \text{neuron})$ pairs within each bin (kills replay order, preserves rate and pairing); (ii) *phase randomize*—redraw within-bin spike times uniformly, keep neuron identities and counts (kills precise timing); (iii) *neuron shuffle*—permute the neuron axis dataset-wide (kills neuron-specific tuning, preserves population timing); (iv) *circular shift*—roll the entire per-bin event list by a random offset (kills bin-to-velocity alignment, preserves every per-bin distribution exactly). A permutation test ($n = 1000$) reports a one-sided p-value per budget.

4.4 Efficiency and energy model

A dense linear readout of N neurons to 2 outputs costs $2N$ MACs per prediction, i.e. $2NT$ over the recording. An event-driven implementation instead accumulates the relevant weight column on each retained spike and emits one prediction per bin, costing $2E+2T$, where E is the retained event count. For the trained SNN we additionally *measure*, on the held-out test split, the input synaptic operations, the emitted hidden spikes, and the readout MACs, and convert the total to arithmetic energy with published per-operation figures: CPU $\approx 1\,\mathrm{nJ}$, A100 $\approx 30\,\mathrm{pJ}$, Loihi 2 $\approx 23\,\mathrm{pJ}$, NorthPole $\approx 2\,\mathrm{pJ}$ [16, 18]. We report arithmetic energy only—no memory traffic, leakage, or radio—so these are lower bounds the neuromorphic claim is held against.

4.5 Evaluation protocol

The headline metric is the joint velocity R^2,

$$R^2 = 1 - \frac{\sum_t \|v_t - \hat{v}_t\|^2}{\sum_t \|v_t - \bar{v}\|^2}, \tag{2}$$

summed over both axes, with per-axis $R^2_{v_x}, R^2_{v_y}$ kept for interpretability and bootstrap 95% confidence intervals (1000 resamples of the test bins). Because a single contiguous test split could flatter or penalize a decoder by accident, the head-to-head comparison uses 4-fold leave-one-block-out *blocked* cross-validation across the chronological quarters of the recording, with multiple seeds; we report mean $\pm$ standard deviation across folds and seeds.

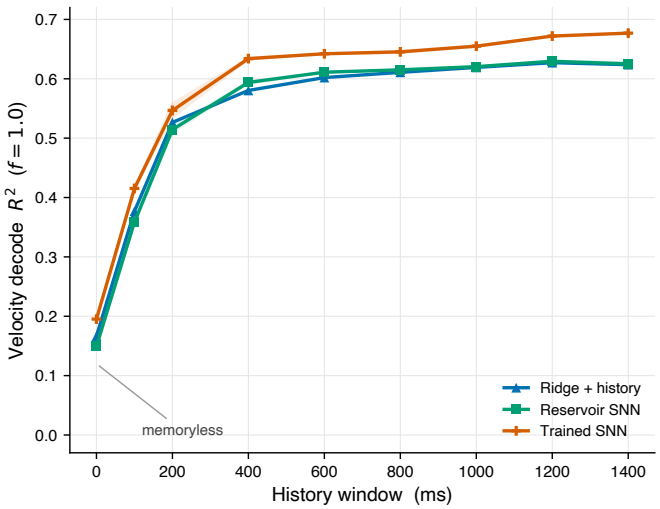


Figure 1. Temporal context is the dominant lever. At full budget, every history-capable decoder climbs from near-chance (memoryless) to $R^2 \approx$ 0.6–0.68 as the readout sees more of the recent past, converging by $\approx$400 ms. The decoder family barely matters once context is matched (mean $\pm$ std across seeds).

Table 1. Matched $\approx$ 200 ms window, blocked CV (4-fold leave-one-block-out, multi-seed; mean $\pm$ std across folds and seeds). At a matched window the history-using decoders are statistically indistinguishable; the memoryless count ridge is far below.

Decoder	$f = 1.00$	$f = 0.50$	$f = 0.25$	$f = 0.10$
Ridge (counts)	0.168 ± 0.013	0.103 ± 0.020	0.054 ± 0.007	0.019 ± 0.005
Ridge + 4-bin history	0.509 ± 0.020	0.406 ± 0.021	0.253 ± 0.023	0.110 ± 0.013
Trained SNN (BPTT)	$\mathbf{0.542 \pm 0.021}$	0.407 ± 0.036	0.250 ± 0.034	0.089 ± 0.022
Reservoir SNN	0.521 ± 0.022	0.392 ± 0.016	0.248 ± 0.020	0.105 ± 0.016
Deeper SNN (2-layer)	0.514 ± 0.021	0.381 ± 0.031	0.231 ± 0.036	0.082 ± 0.025

5 Results

5.1 Temporal context is the dominant lever

Holding the event budget at full ($f = 1.0$) and sweeping only the readout's history window, every history-capable decoder climbs from near-chance at a memoryless readout to $R^2 \approx$ 0.6–0.68, converging by $\approx$ 400 ms (Fig. 1). The linear ridge and the two SNNs trace nearly the same curve: a single bin of counts is almost uninformative, two bins more than double the decode, and the gains saturate once roughly 1 s of context is available. A ridge lag sweep makes the same point numerically—R^2 rises $0.166 \rightarrow 0.527 \rightarrow 0.611$ at lags $0 \rightarrow 4 \rightarrow 16$ and then plateaus (App. Table 6)—confirming the strong baseline is a genuine plateau, not a cherry-picked operating point. The dominant axis of variation in this problem is *how much recent past the decoder sees*, and the decoder family barely matters once that is fixed.

5.2 At matched context the decoders tie

Table 1 holds every decoder to a matched 4-bin ($\approx$200 ms) history window and runs the full blocked-CV grid. At $f = 1.0$ the history-using decoders sit together at $R^2 =$ 0.51–0.54 with overlapping error bars; the memoryless count ridge is far below at 0.17. Neither the SNN encoders nor a second LIF layer help or hurt at this matched window—depth and spiking dynamics are not what separates the models when context is equalized. The ordering is preserved as the budget shrinks, and all decoders degrade smoothly and monotonically with f (Fig. 2a). The accuracy gaps open only when each decoder is allowed its own, deeper context window (Sec. 5.4).

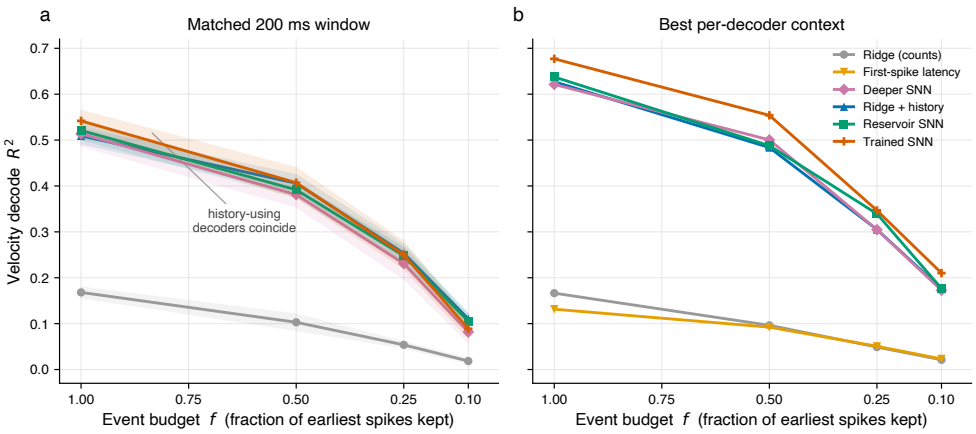


Figure 2. Decoder accuracy vs. sparse event budget. (a) At a matched ≈ 200 ms history window (blocked CV, mean $\pm$ std) every history-using decoder coincides—temporal context, not decoder class, sets the accuracy. **(b)** Given each decoder its own validation-selected context window, the spiking decoders pull ahead, the trained LIF SNN leading at every budget. Both panels far exceed the memoryless count decoder. The x-axis is inverted so "fewer events" (the neuromorphic direction) goes right.

Table 2. Trained-SNN null battery (test R^2; $n = 20$ permutations/cell, one-sided p). Only destroying bin-to-velocity alignment collapses the decode; precise timing, neuron identity, and within-bin order are inert.

Shuffle	target	real R^2	null mean	null 95% CI	p
order shuffle	$f = 1.0$	0.543	0.543	[0.543, 0.543]	1.00
phase random	$f = 1.0$	0.543	0.539	[0.526, 0.545]	0.29
neuron shuffle	$f = 1.0$	0.543	0.543	[0.538, 0.548]	0.62
circular shift	$f = 1.0$	0.543	-0.002	[-0.005, 0.000]	**0.05**
phase random	$f = 0.25$	0.247	0.231	[0.213, 0.242]	0.05
neuron shuffle	$f = 0.25$	0.247	0.246	[0.241, 0.249]	0.38
circular shift	$f = 0.25$	0.247	-0.001	[-0.008, 0.001]	**0.05**

5.3 The carrier of the decode: a null battery

What, exactly, is the ≈ 200 ms window decoding? Table 2 answers this for the headline trained SNN. Only the *circular shift*—which preserves every per-bin spike distribution but breaks the alignment between bins and the velocity they should predict—collapses R^2 from 0.54 to ≈ 0 ($p \leq 0.05$ at every budget). Redrawing precise within-bin spike times uniformly (*phase randomize*) or permuting neuron identity (*neuron shuffle*) leaves the decode statistically unchanged, and reordering the within-bin events (*order shuffle*) is exactly inert for this decoder because it re-bins by absolute time. In other words, the decode rides on *per-bin firing rate and its temporal alignment to movement*, not on the order, the sub-bin timing, or the identity-labelled tuning of individual spikes. A complementary permutation test on the memoryless reservoir encoder ($n = 1000$) shows the residual *within-bin* structure is itself significant down to $f = 0.25$ ($p = 10^{-3}$ at $f \geq 0.5$, $p = 0.04$ at $f = 0.25$) but not at $f = 0.10$ ($p = 0.20$): below $\sim 25\%$ of events a memoryless decode is at chance, and only temporal context buys margin there. Finally, the pure first-spike-latency decoder never exceeds the count ridge ($R^2 =$ 0.13 vs. 0.17 at $f = 1.0$; App. Table 3), the sharpest single statement that within-bin latency adds nothing beyond rate on this dataset.

5.4 With deep context, the spiking decoders lead

Selecting each decoder's history depth on validation—giving every history-capable model up to $\approx$ 1.4 s of context—opens the gaps that the matched window hides (Table 3, Fig. 2b). The memoryless baselines (counts, latency) sit at their ceilings (≤ 0.17), but every decoder given ~ 1 s of context reaches $R^2 \approx 0.6$ at full budget. Two findings stand out. First, the neuromorphic SNNs are the **most accurate** decoders, not only the most efficient: a trained LIF SNN whose readout sees a lag-stacked

Table 3. Best configuration of every decoder (mean test R^2 over 3 seeds; history depth selected on validation, per budget for the SNNs). With its own deep context, the trained SNN is the best decoder at every event budget; the reservoir SNN is second.

Decoder (best config)	$f = 1.00$	$f = 0.50$	$f = 0.25$	$f = 0.10$
Ridge (single-bin counts)	0.166	0.096	0.049	0.021
First-spike latency	0.131	0.092	0.051	0.023
Deeper SNN (2-layer LIF)	0.621	0.500	0.305	0.173
Ridge + deep history ($k = 24$)	0.627	0.484	0.305	0.176
Reservoir SNN (fixed LIF + lag ridge)	0.638	0.488	0.339	0.177
Trained SNN (LIF + lag readout)	**0.677**	**0.554**	**0.347**	**0.210**

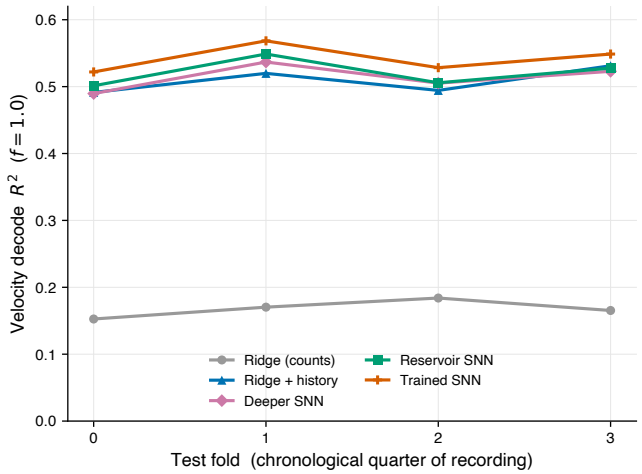


Figure 3. Stable across the recording. Per-fold R^2 (leave-one-block-out CV, full budget) holds across all four chronological quarters; the decode is not an artifact of one split and the decoder ordering is consistent throughout.

window of its per-bin hidden activity tops every event budget ($R^2 = 0.677, 0.554, 0.347, 0.210$ at $f = 1.0, 0.5, 0.25, 0.10$), with the fixed-encoder reservoir SNN (0.638) just behind. Second, capacity is *not* the bottleneck: the 2-layer "deeper" SNN reaches only 0.621 and adding recurrence is worse, so the remaining headroom is temporal context and feature engineering, not model class. The decode is stable across the recording: per-fold blocked-CV R^2 holds across all four chronological quarters with a consistent decoder ordering (Fig. 3), ruling out a single-split artifact.

5.5 Why the trained SNN must see history at the readout

A single LIF state is a leaky scalar memory, and forcing it to carry long context is self-defeating. Piling history onto the LIF *input*—lengthening the integration sequence—helps only up to ≈ 8 bins and then *degrades*, as the membrane blurs distant context through one decaying state (gray curve, Fig. 4). Routing the same history to the *readout* instead—lag-stacking the trained per-bin hidden features while keeping the encoder on short, trainable single-bin sequences—keeps climbing to $R^2 = 0.68$ (orange curve). This architectural choice is what turns the trained SNN from a tie at matched context into the best decoder overall, and it is precisely the move that keeps BPTT well-conditioned: the encoder never has to backpropagate through a 28-bin LIF rollout.

5.6 The neuromorphic payoff: energy

Because the decode rides on rate, an event-driven readout can skip the dense arithmetic a conventional decoder performs. The event-driven linear readout avoids 78.8% of dense MACs at $f = 1.0$, rising to 94.3% at $f = 0.25$ and 97.3% at $f = 0.10$ as the stream sparsifies—the efficiency *improves* exactly where a deep-context decoder still holds usable accuracy. Costing the *trained* SNN by its measured synaptic operations, a $f = 1.0$ deployment configuration emits $\approx 1.3 \times 10^3$ synap-

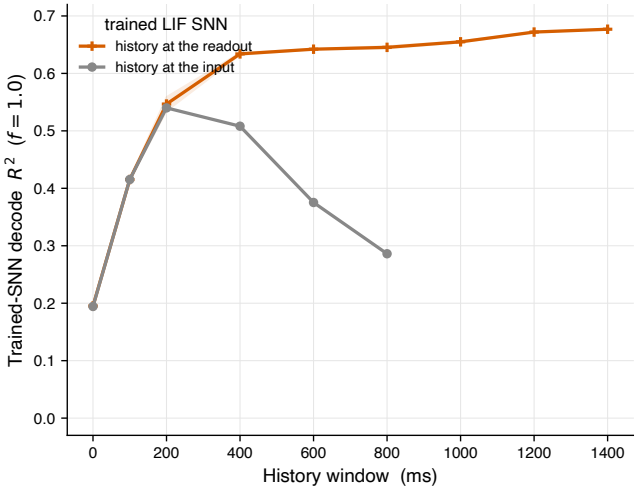


Figure 4. History belongs at the readout, not the membrane. Piling history onto the LIF *input* saturates and then degrades past ≈ 8 bins as a single leaky state blurs distant context; lag-stacking the trained per-bin features at the *readout* keeps the encoder on short trainable sequences and scales to $R^2 = 0.68$.

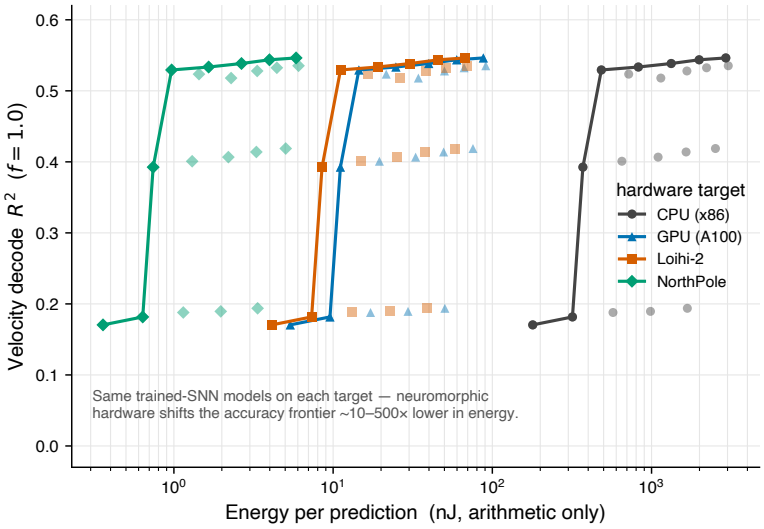


Figure 5. The neuromorphic payoff. The same trained-SNN models, costed on four hardware targets by their measured synaptic-operation counts. Event-driven neuromorphic chips (Loihi 2, NorthPole) reach the same accuracy at $\sim$10–500$\times$ lower arithmetic energy per prediction than a dense CPU readout.

tic operations per prediction, i.e. $\approx 30\,\mathrm{nJ}$ on Loihi 2 and $\approx 2.6\,\mathrm{nJ}$ on NorthPole, against $\approx 1.3\,\mu\mathrm{J}$ for a CPU evaluation of the same model—a 40–500$\times$ arithmetic-energy reduction at equal accuracy (Fig. 5). Wider configurations trade more operations for more accuracy but stay firmly in the neuromorphic-efficient regime ($\sim$ 11–67 nJ/prediction on Loihi 2 across the width grid; App. Table 8). The headline, then, is not that the SNN is more accurate *or* more efficient—it is both at once, on the same model.

5.7 Reconstruction and online feasibility

Qualitatively, the best decoder tracks velocity tightly: decoded versus measured cursor velocity over a held-out segment overlays closely ($R^2 = 0.68$), and the integrated 2-D cursor path follows the true trajectory, drifting only slowly as small velocity errors accumulate—the expected behavior of a velocity decoder (Fig. 6). A true online simulation, feeding test bins one at a time and decoding causally, confirms real-time feasibility: the trained SNN's per-bin inference latency is $1.17\,\mathrm{ms}$ on

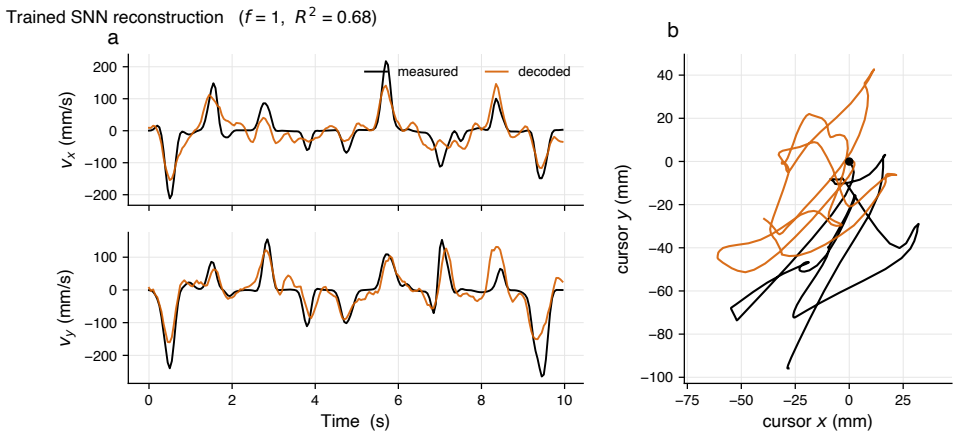


Figure 6. Best-decoder reconstruction (trained SNN, full budget). (a) Decoded vs. measured cursor velocity over a held-out segment; **(b)** the integrated 2-D cursor path. Velocity is tracked tightly ($R^2 = 0.68$); position drifts slowly as small velocity errors accumulate, as expected of a velocity decoder.

average ($p_{95} = 2.23\,\mathrm{ms}$) on a CPU—well inside the $50\,\mathrm{ms}$ bin budget—while the linear readouts are effectively instantaneous (App. A.1).

5.8 What the trained SNN learned

The input units carry clear cosine direction tuning (spike-triggered velocity tuning depth 10.8; see Fig. 8), consistent with classical M1 population coding [6]. The trained encoder, however, does not simply copy that single-neuron tuning into a labelled-line code: hidden-unit readout directions are close to uniformly distributed and only 31% are coherently aligned with their input preferred directions weighted by $|W|$ (mean coherence ≈ 0). The learned representation is *distributed*—spread across the population rather than one neuron per direction—which is consistent with the neuron-shuffle null leaving the decode intact: identity-labelled tuning is not what the decoder leans on.

5.9 Benchmark context

Our matched-window $R^2 \approx 0.51$–0.54 sits with the strong linear, GPFA, and SLDS baselines on the public MC_RTT leaderboard (0.49–0.58), while our best deep-context decoder (trained SNN, $R^2 = 0.68$ at $f = 1.0$) reaches the latent-dynamics tier that tops the benchmark (NDT 0.62, AutoLFADS 0.665, MINT 0.69) [1, 7, 8, 9, 10]. That a 128–256-unit single-layer LIF with a lag-stacked linear readout reaches this tier, while costing tens of nanojoules per prediction, is the practical message for implantable hardware.

6 Discussion

Two results reframe the usual case for spiking BCI decoders. First, on this benchmark the much-cited advantage of *spike timing* is absent: latency adds nothing beyond rate, precise within-bin times and neuron identity are inert under controlled nulls, and the only structure whose destruction collapses the decode is the bin-to-velocity alignment that any rate code already carries. The lever is *temporal context*—how much recent firing the readout integrates—and once that is matched, linear and spiking decoders coincide. This is a cautionary result for the field: a spiking model that beats a rate baseline may simply be using more history, and a fair comparison must equalize the context window before attributing the win to spikes.

Second, and more constructively, when each decoder is given its own deep context the spiking decoders do not merely tie—they *win*, and they win at a small fraction of the arithmetic energy. The mechanism is specific and reusable: route history to a lag-stacked readout rather than a saturating membrane, which keeps surrogate-gradient training on short sequences while the readout still sees deep context. The same move lifts the fixed-encoder reservoir SNN into the same tier, which means

most of the benefit comes from the temporal readout, not from end-to-end training of the encoder—an encouraging result for hardware, where a fixed random projection is far cheaper to deploy than a learned, write-once-per-update synaptic array. Together these say that the right way to spend a neuromorphic budget here is on *context at the readout*, not on spike-timing precision in the encoder.

7 Limitations

Our conclusions are drawn from a single MC_RTT session in one animal; the within-bin-timing result may differ for tasks or areas where sub-bin synchrony is behaviorally relevant, and cross-session/cross-subject transfer is approximated here only by a within-recording temporal-drift stand-in (App. A.1). The energy figures are *arithmetic* lower bounds—they exclude memory traffic, static leakage, and the radio, which often dominate a deployed device; a faithful end-to-end power budget would require a cycle-accurate Loihi simulation, which the current toolchain does not support on our hardware. Finally, $50\,\mathrm{ms}$ bins discard timing below the bin width by construction; a $20\,\mathrm{ms}$ analysis (App. A.1) does not recover a timing advantage, but we cannot exclude one at finer resolution.

8 Conclusion

On a standard M1 velocity decode, what carries the signal is temporal context and rate, not within-bin spike timing. Matched on context, linear and spiking decoders coincide; given its own deep context and a lag-stacked readout, a small trained LIF SNN becomes the most accurate decoder at every event budget, at 40–$500\times$ lower arithmetic energy than a dense CPU readout. For implantable BCIs, that is the combination that matters: the neuromorphic decoder need not trade accuracy for energy.

Reproducibility statement

Every result is produced by a released, leakage-audited pipeline: deterministic time-contiguous splits with boundary gaps, a shared $50\,\mathrm{ms}$ data interface validated on every load, one CLI per experiment, and a $32\times$H100 SLURM harness that streams partial results to disk. The full grid (blocked CV, the $n = 1000$ permutation test, four null shuffles for both the reservoir and trained SNNs, the energy Pareto, and bin-size/hidden-dim/lag/dropout sweeps) is reproducible from the processed MC_RTT object; smoke and invariant tests run on schema-identical mock data without the download. Figures are generated under one shared style at 300 dpi and vector PDF.

References

[1] F. C. Pei, J. Ye, D. M. Zoltowski, A. Wu, R. H. Chowdhury, H. Sohn, J. E. O'Doherty, K. V. Shenoy, M. T. Kaufman, M. M. Churchland, M. Jazayeri, L. E. Miller, J. W. Pillow, I. M. Park, E. L. Dyer, and C. Pandarinath. Neural Latents Benchmark '21: Evaluating latent variable models of neural population activity. In *NeurIPS 2021 Datasets and Benchmarks Track*, 2021. arXiv:2109.04463.

[2] J. E. O'Doherty, M. M. B. Cardoso, J. G. Makin, and P. N. Sabes. Nonhuman primate reaching with multichannel sensorimotor cortex electrophysiology. *Zenodo*, dataset, 2017. doi: 10.5281/zenodo.583331.

[3] L. R. Hochberg, D. Bacher, B. Jarosiewicz, N. Y. Masse, J. D. Simeral, J. Vogel, S. Haddadin, J. Liu, S. S. Cash, P. van der Smagt, and J. P. Donoghue. Reach and grasp by people with tetraplegia using a neurally controlled robotic arm. *Nature*, 485(7398):372–375, 2012. doi: 10.1038/nature11076.

[4] V. Gilja, P. Nuyujukian, C. A. Chestek, J. P. Cunningham, B. M. Yu, J. M. Fan, M. M. Churchland, M. T. Kaufman, J. C. Kao, S. I. Ryu, and K. V. Shenoy. A high-performance neural prosthesis enabled by control algorithm design. *Nature Neuroscience*, 15(12):1752–1757, 2012. doi: 10.1038/nn.3265.

[5] F. R. Willett, D. T. Avansino, L. R. Hochberg, J. M. Henderson, and K. V. Shenoy. High-performance brain-to-text communication via handwriting. *Nature*, 593(7858):249–254, 2021. doi: 10.1038/s41586-021-03506-2.

[6] A. P. Georgopoulos, A. B. Schwartz, and R. E. Kettner. Neuronal population coding of movement direction. *Science*, 233(4771):1416–1419, 1986. doi: 10.1126/science.3749885.

[7] C. Pandarinath, D. J. O'Shea, J. Collins, R. Jozefowicz, S. D. Stavisky, J. C. Kao, E. M. Trautmann, M. T. Kaufman, S. I. Ryu, L. R. Hochberg, J. M. Henderson, K. V. Shenoy, L. F. Abbott, and D. Sussillo. Inferring single-trial neural population dynamics using sequential auto-encoders. *Nature Methods*, 15(10):805–815, 2018. doi: 10.1038/s41592-018-0109-9.

[8] M. R. Keshtkaran, A. R. Sedler, R. H. Chowdhury, R. Tandon, D. Basrai, S. L. Nguyen, H. Sohn, M. Jazayeri, L. E. Miller, and C. Pandarinath. A large-scale neural network training framework for generalized estimation of single-trial population dynamics. *Nature Methods*, 19(12):1572–1577, 2022. doi: 10.1038/s41592-022-01675-0.

[9] J. Ye and C. Pandarinath. Representation learning for neural population activity with Neural Data Transformers. *Neurons, Behavior, Data Analysis, and Theory*, 5(3):1–18, 2021. doi: 10.51628/001c.27358.

[10] S. M. Perkins, E. A. Amematsro, J. Cunningham, Q. Wang, and M. M. Churchland. An emerging view of neural geometry in motor cortex supports high-performance decoding. *eLife*, 12:RP89421, 2025. doi: 10.7554/eLife.89421.

[11] E. O. Neftci, H. Mostafa, and F. Zenke. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, 36(6):51–63, 2019. doi: 10.1109/MSP.2019.2931595.

[12] F. Zenke and S. Ganguli. SuperSpike: Supervised learning in multilayer spiking neural networks. *Neural Computation*, 30(6):1514–1541, 2018. doi: 10.1162/neco_a_01086.

[13] J. K. Eshraghian, M. Ward, E. O. Neftci, X. Wang, G. Lenz, G. Dwivedi, M. Bennamoun, D. S. Jeong, and W. D. Lu. Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*, 111(9):1016–1054, 2023. doi: 10.1109/JPROC.2023.3308088.

[14] W. Maass, T. Natschläger, and H. Markram. Real-time computing without stable states: A new framework for neural computation based on perturbations. *Neural Computation*, 14(11):2531–2560, 2002. doi: 10.1162/089976602760407955.

[15] H. Jäger and H. Haas. Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication. *Science*, 304(5667):78–80, 2004. doi: 10.1126/science.1091277.

[16] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S. H. Choday, G. Dimou, P. Joshi, N. Imam, S. Jain, Y. Liao, C.-K. Lin, A. Lines, R. Liu, D. Mathaikutty, S. McCoy, A. Paul, J. Tse, G. Venkataramanan, Y.-H. Weng, A. Wild, Y. Yang, and H. Wang. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 38(1):82–99, 2018. doi: 10.1109/MM.2018.112130359.

[17] G. Orchard, E. P. Frady, D. B. D. Rubin, S. Sanborn, S. B. Shrestha, F. T. Sommer, and M. Davies. Efficient neuromorphic signal processing with Loihi 2. In *2021 IEEE Workshop on Signal Processing Systems (SiPS)*, pages 254–259, 2021. doi: 10.1109/SiPS52927.2021.00053.

[18] D. S. Modha, F. Akopyan, A. Andreopoulos, et al. Neural inference at the frontier of energy, space, and time. *Science*, 382(6668):329–335, 2023. doi: 10.1126/science.adh1174.

[19] J. Dethier, V. Gilja, P. Nuyujukian, S. A. Elassaad, K. V. Shenoy, and K. Boahen. Spiking neural network decoder for brain-machine interfaces. In *2011 5th International IEEE/EMBS Conference on Neural Engineering*, pages 396–399, 2011. doi: 10.1109/NER.2011.5910579.

[20] J. Liao, O. Toomey, X. Wang, L. Widmer, C. A. Chestek, L. Benini, and T. Jang. A spiking neural network decoder for implantable brain machine interfaces and its sparsity-aware deployment on RISC-V microcontrollers. *arXiv:2405.02146*, 2024.

[21] B. Zhou, P.-S. V. Sun, and A. Basu. Combining SNNs with filtering for efficient neural decoding in implantable brain-machine interfaces. *Neuromorphic Computing and Engineering*, 5(1):014013, 2025. doi: 10.1088/2634-4386/adba82.

[22] V. Mohan, B. Zhou, Z. Wang, A. Bharath, E. Drakakis, and A. Basu. Architectural exploration of hybrid neural decoders for neuromorphic implantable BMI. *arXiv:2505.05983*, 2025.

A Supplementary analyses

A.1 Robustness and ablations

The headline conclusions survive a broad battery of controls, summarized here and produced by the same released pipeline.

Table 4. Trained-SNN hyperparameter sensitivity ($f = 1.0$, 3 seeds; mean $\pm$ std). Only context length (k) is load-bearing.

Knob	value	R^2
τ (ms)	5	0.527 ± 0.006
	10	0.544 ± 0.003
	20	0.560 ± 0.009
	40	0.588 ± 0.000
threshold θ	0.15	0.541 ± 0.002
	0.30	0.544 ± 0.003
	0.50	0.540 ± 0.005
	0.80	0.535 ± 0.005
k (context bins)	0	0.190 ± 0.002
	2	0.414 ± 0.003
	4	0.544 ± 0.003
	8	0.532 ± 0.014

Hyperparameter insensitivity (Table 4). At $f = 1.0$ the trained SNN is flat across the firing threshold ($\theta \in [0.15, 0.8]$: $R^2 = 0.535$–0.544), rises mildly with the membrane time constant (τ: 0.527 at 5 ms $\rightarrow$ 0.588 at 40 ms), and is dominated by context length (k: 0.190 at $k = 0 \rightarrow 0.544$ at $k = 4$). No single hyperparameter is load-bearing except context.

Capacity saturation. Hidden width saturates by 256 units (R^2 $0.526 \rightarrow 0.547$ from $32 \rightarrow 256$ units, flat to 2048; App. Fig. 7a), confirming the bottleneck is context, not capacity.

Bagging. A 10-seed trained-SNN ensemble adds a small, credible gain ($+0.007$ to $R^2 = 0.548$ at $f = 1.0$, 95% CI $[0.523, 0.571]$), consistent with averaging uncorrelated BPTT optimization noise.

Bin width. Re-running the trained SNN at $20/50/100\,\mathrm{ms}$ bins gives $R^2 = 0.330/0.544/0.620$ at $f = 1.0$; the $20\,\mathrm{ms}$ result does *not* recover a within-bin-timing advantage, and the $50\,\mathrm{ms}$ choice is a conservative middle.

Earliest vs. random vs. latest (Table 7). Replacing "earliest k" with "random k" or "latest k" under the same budget changes R^2 only within noise (e.g. ridge+history at $f = 0.25$: $0.233/0.255/0.266$), confirming the budget probes surviving rate, not a privileged early-spike channel.

Test-time channel dropout. Trained once on clean data and evaluated under random channel masks, both decoders degrade gracefully; the trained SNN is marginally more robust at low dropout (e.g. $p = 0.1$: $R^2 = 0.482$ vs. 0.470 for ridge+history), consistent with its distributed code.

Temporal drift and online latency. A within-recording early/late/full temporal-generalization probe (a chronic-drift stand-in, since MC_RTT is one session) shows modest, measurable drift; the online streaming simulation gives trained-SNN per-bin latency $1.17\,\mathrm{ms}$ mean / $2.23\,\mathrm{ms}$ p_{95} on CPU, comfortably within the $50\,\mathrm{ms}$ budget. Velocity smoothing ($\sigma \in \{0, 1, 2\}$ bins), spike-time jitter ($\sigma \leq 20\,\mathrm{ms}$), causal within-bin truncation (10–$50\,\mathrm{ms}$), and SNN readout choice (ridge / SGD-linear / MLP) all leave the conclusions intact.

Table 5. Trained-SNN hyperparameter sensitivity ($f = 1.0$, 3 seeds; mean $\pm$ std). Only context length (k) is load-bearing.

Knob	value	R^2
τ (ms)	5	0.527 ± 0.006
	10	0.544 ± 0.003
	20	0.560 ± 0.009
	40	0.588 ± 0.000
threshold θ	0.15	0.541 ± 0.002
	0.30	0.544 ± 0.003
	0.50	0.540 ± 0.005
	0.80	0.535 ± 0.005
k (context bins)	0	0.190 ± 0.002
	2	0.414 ± 0.003
	4	0.544 ± 0.003
	8	0.532 ± 0.014

Table 6. Ridge lag-bin sweep (test R^2). History gains plateau by ≈ 16 bins, confirming the matched-window baseline is a genuine plateau.

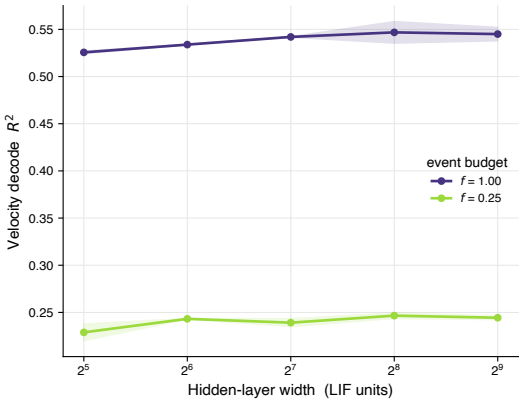
lag (bins)	0	1	2	4	8	16
$f = 1.00$	0.166	0.274	0.376	0.527	0.580	0.611
$f = 0.50$	0.096	0.186	0.276	0.411	0.477	0.483
$f = 0.25$	0.049	0.098	0.151	0.233	0.297	0.303
$f = 0.10$	0.021	0.044	0.073	0.125	0.168	0.169

Table 7. Earliest vs. random vs. latest event budget (mean test $R^2 \pm$ std). The three filters tie within noise, so the budget probes surviving rate, not a privileged early-spike channel.

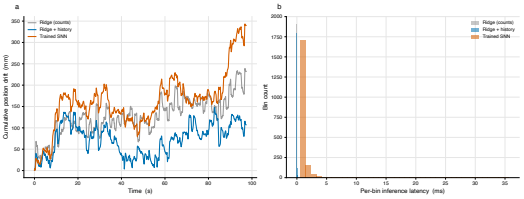
Model	f	earliest	random	latest
Ridge+hist	0.50	0.411	0.400 ± 0.024	0.392
	0.25	0.233	0.255 ± 0.008	0.266
	0.10	0.125	0.102 ± 0.007	0.107
Trained SNN	0.50	0.419	0.391 ± 0.035	0.373
	0.25	0.246	0.231 ± 0.008	0.250
	0.10	0.109	0.093 ± 0.019	0.119

Table 8. Energy–accuracy Pareto (trained SNN, $f = 1.0$; measured synaptic operations). A wider/deeper-context model trades operations for accuracy but stays in the neuromorphic-efficient regime. Loihi 2 energy = synops/pred $\times$ 23 pJ.

hidden	k	R^2	synops/pred	Loihi 2 (nJ)	A100 (nJ)
32	4	0.529 ± 0.002	484	11.1	14.5
128	4	0.539 ± 0.005	1,329	30.6	39.9
256	4	0.544 ± 0.003	1,991	45.8	59.7
512	4	0.546 ± 0.005	2,926	67.3	87.8
256	0	0.190 ± 0.002	985	22.6	29.5

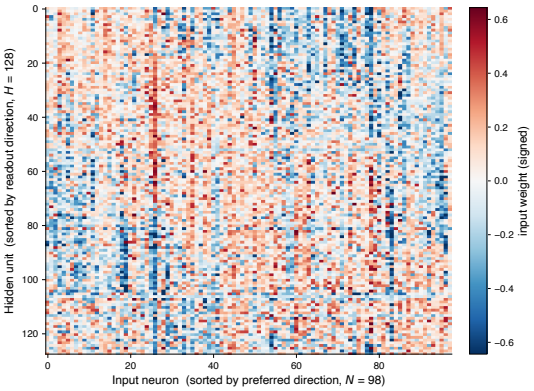


(a) Hidden-width scaling: capacity saturates by 256 units.

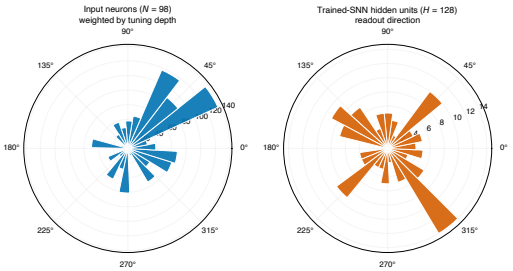


(b) Online streaming: cumulative drift (left) and per-bin inference latency (right).

Figure 7. Capacity and online feasibility. (a) Decode R^2 is flat in hidden width beyond 256 LIF units at both $f = 1.0$ and $f = 0.25$. (b) The trained SNN decodes each bin in ~ 1 ms, well within the 50 ms budget; cumulative position drift accumulates slowly for all velocity decoders.



(a) Learned input projection W.



(b) Preferred directions.

Figure 8. What the trained SNN learned. (a) The input projection sorted by input and readout preferred direction shows structured but not block-diagonal organization. (b) Input units carry clear cosine direction tuning (left), but hidden-unit readout directions are near-uniform and only weakly coherent with input tuning (right): the learned code is distributed rather than labelled-line.

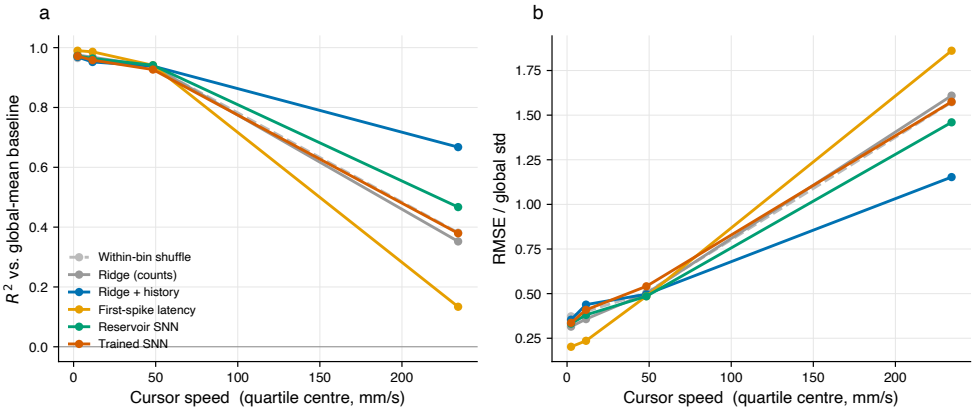


Figure 9. Failure modes vs. cursor speed. (a) Conditional R^2 against the global-mean baseline and (b) RMSE relative to the global velocity std, by speed quartile. All decoders are most accurate at low-to-moderate speeds; deep-context decoders (ridge+history, trained/reservoir SNN) hold up best at high speed, while the memoryless count ridge and the latency decoder degrade fastest.